



Ferramentas de Assessoria

Documentação Técnica

Arquitetura, stack e decisões de implementação

Versão 2.2.0 · Julho de 2026

Gustavo Meireles — Assessoria de Investimentos XP

SUMÁRIO



1

VISÃO GERAL

o que o app faz

2

STACK TECNOLÓGICA

3

ESTRUTURA DO PROJETO

4

FLUXO PRINCIPAL — ABA RENTABILIDADE

5

DECISÕES E ARMADILHAS DOCUMENTADAS

6

BUILD E TESTES

7

IDENTIDADE VISUAL

1 VISÃO GERAL

Aplicativo desktop Windows que substitui a ferramenta antiga em Python — um conjunto de utilitários para assessores de investimento: leitura automatizada de relatórios XP, mensagens de rentabilidade, geração de e-mails de ordem, calculadoras financeiras e edição de PDF.

ABA	O QUE FAZ
Rentabilidade	Lê os relatórios XPerformance da pasta, monta a mensagem de rentabilidade (mês/ano/12 meses) por cliente.
E-mails de Ordem	Monta o texto padronizado de ordem por produto (Renda Fixa, COE, Erro Operacional...).
Tabela de Deságio	Calcula e desenha a tabela de ágio/deságio de títulos como imagem.
Editor de PDF	Anota (desenho, texto, tarja, imagem, assinatura) um PDF existente.
Imagens em PDF	Junta N imagens escolhidas pelo usuário num único PDF novo.
Apresentação	Modo slideshow de um PDF/apresentação carregada, tela cheia.
Typeform	Formulário de coleta de dados do cliente durante a reunião.
Calculadora / Comparadora / Previdenciária / Compromissada / Aposentadoria	Calculadoras financeiras independentes, sem dependência dos relatórios.

2 STACK TECNOLÓGICA

CAMADA	TECNOLOGIA	POR QUÊ
Backend	Go + Wails v2	binário nativo único, sem runtime externo pro usuário instalar
Frontend	JavaScript vanilla (sem framework)	app pequeno o bastante pra não precisar de build tooling extra
Leitura de PDF	go-pdfium (WASM via wazero)	extraí texto/renderiza páginas sem depender de cgo
Escrita de PDF	github.com/go-pdf/fpdf	monta PDF novo a partir de imagens (Editor de PDF e Imagens em PDF)
Banco de dados	SQLite (modernc.org/sqlite)	um arquivo por pasta de relatórios, sem servidor, sem cgo
Configuração	JSON em %APPDATA%	simples de inspecionar e fazer backup

POR QUE SEM CGO?

cgo exige um compilador C configurado na máquina que builda o app — go-pdfium (WASM) e modernc.org/sqlite (Go puro) evitam essa dependência, então "go build" funciona em qualquer máquina com Go instalado, sem toolchain C.

3 ESTRUTURA DO PROJETO

```
rentabilidade/
├─ main.go           # entrypoint, redireciona stdio, inicia o Wails
├─ app.go            # métodos expostos ao frontend (bindings Wails)
├─ dto.go            # structs "wire" (JSON-safe) expostas ao frontend
├─ config.go         # struct de preferências + leitura/escrita do config.json
├─ internal/
│   ├─ pdfreport/    # extração de dados dos relatórios XP (regex sobre o texto do PDF)
│   ├─ rentabilidade/ # banco SQLite, placeholders da mensagem, exportação CSV
│   ├─ clientdb/     # leitura da base de clientes (CSV)
│   ├─ emailgen/     # catálogo de categorias, monta o texto de e-mail de ordem
│   ├─ pdfedit/      # monta um PDF novo a partir de páginas/imagens (fpdf)
│   ├─ assinaturas/  # imagens de assinatura salvas (Configurações → Assinatura)
│   └─ whatsapp/     # monta o link wa.me a partir do telefone
├─ frontend/
│   ├─ src/
│   │   ├─ main.js   # monta a sidebar, o roteamento entre abas, Configurações
│   │   ├─ state.js  # estado em memória compartilhado entre as abas
│   │   └─ tabs/     # um módulo por aba (mount(container, ctx))
│   │       └─ ui/    # componentes (el/btn/...), ícones SVG, theme.css
│   └─ wailsjs/      # bindings gerados (não editar à mão)
└─ build/
    ├─ appicon.png   # ícone-fonte, 1024×1024
    └─ windows/icon.ico # gerado a partir do appicon.png
```

4 FLUXO PRINCIPAL — ABA RENTABILIDADE



4.1 EXTRAÇÃO DOS DADOS (INTERNAL/PDFREPORT)

O texto extraído do PDF passa por uma normalização (PDFium devolve U+00A0 em vez de espaço comum em vários pontos do relatório — sem isso os regexes não batem).

- Código da conta é obrigatório — sem ele o arquivo vira uma falha de processamento.
- As linhas **MÊS**, **ANO** e **12M** da tabela "RESUMO DE INFORMAÇÕES DA CARTEIRA" são lidas pelo mesmo padrão de regex genérico (`padraoLinhaResumo(rotulo)`), só trocando o rótulo — as três são obrigatórias (arquivo sem alguma delas vira falha de processamento).
- Data de referência é opcional (usada pra desempate quando há PDFs duplicados do mesmo código).
- Patrimônio total bruto é opcional — usado só pra ordenar a lista de clientes.

4.2 DEDUPLICAÇÃO POR CÓDIGO

Quando duas leituras da pasta encontram arquivos diferentes com o mesmo código de conta (relatório baixado de novo, arquivo renomeado), só o de Data de Referência mais recente é devolvido pela lista de registros — o(s) mais antigo(s) fica(m) de fora por completo, inclusive da exportação CSV.

4.3 A LISTA DE CLIENTES (LISTARCLIENTES)

SITUAÇÃO	ONDE APARECE
Código da base com relatório processado	Linha normal, com os dados do relatório
Código da base sem relatório (ou PDF com falha)	Linha em branco, ordenada por nome
Código de um relatório sem cliente correspondente na base	Linha extra no fim, só com o código

4.4 MENSAGEM FINAL

`modelo_mensagem.txt` (um por pasta) guarda o texto com os placeholders. `ValoresPlaceholder` resolve cada um: `_Rent`, `_RentA`, `_Rent12M`, `_Perc`, `_PercA`, `_Perc12M`, `_CDI`, `_CDIA`, `_CDI12M`, `_Nome` — os valores em R\$/% já vêm formatados em pt-BR pelo backend (`pdfreport.FormatarReais` / `FormatarPercentual`), fonte única de verdade reaproveitada também pelo frontend (`RegistroDTO` carrega os campos já formatados, pra não duplicar a lógica em JS). "Copiar mensagem"/"Enviar WhatsApp" usam esse texto final; o WhatsApp abre via link `wa.me`.

5 DECISÕES E ARMADILHAS DOCUMENTADAS

Registro do "porquê" por trás de escolhas não óbvias — pra não redescobrir o mesmo problema duas vezes.

Redirecionamento de stdio é obrigatório

O .exe é uma aplicação GUI (sem console) — nesse caso o Windows não dá ao processo um handle válido de stdout/stderr. Bibliotecas que escrevem direto em os.Stdout (go-pdfium, wazero) travam/erram sem isso. `main.go` redireciona os.Stdout/os.Stderr pra um arquivo de log antes de qualquer outra coisa rodar.

Cache de compilação do WASM

Sem `WithCompilationCache`, o wazero recompila o binário WASM do PDFium do zero toda vez que o app abre (~3,4s medido, interface "travada" na abertura). Com cache persistente em disco, só a primeira execução em cada máquina paga esse custo.

PDFium inicializa sob demanda

O motor de PDF só é criado quando a primeira operação que precisa dele acontece (varrer uma pasta, abrir o Editor de PDF) — abrir o app sem nenhuma pasta configurada não paga esse custo de inicialização.

A ordem do texto extraído do PDF não é a ordem visual da página

PDFium devolve os "runs" de texto na ordem em que foram desenhados no PDF, que raramente bate com a ordem de leitura visual (colunas, tabelas). Os regexes de extração dependem de rótulos únicos e formato fixo (ver 4.1) justamente por isso — não dá pra confiar em posição/coordenada.

Instância única do aplicativo

Um segundo clique no .exe traz a janela existente pra frente em vez de abrir outra — evita duas cópias do programa disputando o mesmo arquivo SQLite ao mesmo tempo.

Migração de banco sem perda de dados

Colunas novas em `rentabilidades.db` (ex.: as 3 do 12M) entram via `ALTER TABLE` idempotente na abertura do banco — erro de "coluna já existe" é esperado e ignorado, então bancos antigos ganham a coluna nova sem perder o que já tinha.

"R\$" automático nos campos de valor (emailgen.Field.Moeda)

Campos de e-mail marcados `Moeda: true` recebem o prefixo "R\$" automaticamente em `ResolveValues` se o usuário não tiver digitado — só quando o campo não está vazio (não mexe no fallback "[Label]" nem em campo opcional vazio, e não duplica "R\$" se o usuário já tiver digitado). Só existe pra campos que são exclusivamente um valor monetário — campos que aceitam quantidade ou texto livre (ex.: "Resgate Total") ficam de fora de propósito.

EmailCompleto agora recebe o código da conta

`Category.EmailCompleto` (usado por Resgate Prev e Erro Operacional — modelos com texto 100% próprio, sem a introdução/fechamento padrão) só recebia o nome do cliente. O modelo "Erro Operacional" precisa também do código da conta na linha "Cliente: código - nome", então a assinatura passou a ser `func(codigo, nome string, v map[string]string) string` — e de quebra, usar `EmailCompleto` já garante "uma ordem por e-mail" de graça (mesma trava que Resgate Prev usa), sem precisar de nenhuma flag nova no catálogo.

6 BUILD E TESTES

```
cd rentabilidade
wails build          # gera build/bin/FerramentasAssessoria.exe
wails dev            # roda com hot-reload do frontend
go test ./...        # testes de todos os pacotes Go
```

Os testes de integração contra um PDF real (extração completa, incluindo 12M) são pulados por padrão — apontar a variável de ambiente roda o fluxo completo:

```
PDFREPORT_TEST_PDF=<caminho do PDF> go test ./internal/... -v
```

A versão em `wails.json` (campo "info") fica gravada nas propriedades do .exe — é o que aparece em Configurações → Sobre e no botão direito → Propriedades → Detalhes do Windows, o que também ajuda a passar confiança em máquinas onde o SmartScreen desconhece o binário.

7 IDENTIDADE VISUAL

O ícone é um "raiado" (sunburst): um leque de raios dourados convergindo para um ponto de luz, sobre um quadrado arredondado verde-pinho — remete tanto a um fecho de luz guiando o assessor quanto ao gráfico de rentabilidade "para cima e para a direita".



Ícone em 1024×1024 — a mesma imagem-base de todos os tamanhos usados no Windows.

Todos os tamanhos (16 a 1024px, incluindo o .ico multi-resolução do Windows e a versão de 160px da barra lateral) são gerados programaticamente a partir dessa única imagem-fonte, com supersampling — sem depender de nenhuma ferramenta de design externa.

- build/appicon.png — fonte única, 1024×1024
- build/windows/icon.ico — multi-resolução, gerado a partir do appicon.png
- Versão de 160px — usada no cabeçalho expandido da barra lateral